

Mathematical Formulation of the Quad Persona System in the 17-Axis Architecture

Overview and Knowledge Space Mapping

The **Quad Persona System** is now formally grounded in the **17-Axis Universal Knowledge Graph (UKG)** architecture. We represent the knowledge space Ω as a 17-dimensional space $\Omega = X_1 \times X_2 \times \dots \times X_{17}$, where each X_i corresponds to one axis of the UKG. In practical terms, any knowledge item or query is mapped to a 17-part coordinate $\mathbf{x} = (x_1, x_2, \dots, x_{17})$ covering all facets (domains, context, meta-evaluations) ¹ ². For example, a node may be indexed with:

- **Axis 1 (Pillar):** fundamental domain or discipline (e.g. *Religious Texts* vs. *Physics*) ¹
- **Axis 2 (Sector/Industry):** application sector or NAICS domain (e.g. *Defense* or *Healthcare*)
- **Axis 3 (Honeycomb):** semantic cross-domain links (e.g. *Bible* ↔ *Quran* ↔ *Torah* indicating related corpora) ¹
- **Axis 4 (Branch):** hierarchical sub-domain or process lineage (a Nuremberg-coded taxonomy path)
- **Axis 5 (Node):** atomic knowledge unit (e.g. a specific fact, verse, or component ID) ³
- **Axis 6 (Spiderweb):** regulatory or equivalent cross-references (e.g. mapping *Canon Law* ↔ *Civil Law*) ⁴
- **Axis 7 (Octopus):** aggregated multi-source context (e.g. *Vatican* ↔ *Protestant* ↔ *Orthodox* sources) ⁴
- **Axis 8 (Knowledge Persona Role):** epistemic lens – the **Knowledge Expert** persona's perspective ⁵
- **Axis 9 (Sector Persona Role):** practitioner's lens – the **Sector Expert** persona perspective ⁵
- **Axis 10 (Regulatory Persona Role):** governance lens – the **Regulatory Expert** persona ⁶
- **Axis 11 (Compliance Persona Role):** audit lens – the **Compliance Expert** persona ⁶
- **Axis 12 (Location):** spatial or jurisdictional context (e.g. *US.DC* or *Global*) ⁶
- **Axis 13 (Time):** temporal context (timestamp or interval, e.g. *2025.Q4*) ⁶
- **Axis 14 (Confidence):** knowledge confidence/provenance score (0–1 or categorical confidence level) ²
- **Axis 15 (Risk/Impact):** risk or uncertainty level associated (e.g. *High* risk vs. numeric) ²
- **Axis 16 (Ethics/Compliance Tag):** ethical alignment or compliance status (e.g. *Compliant* under certain law) ⁷
- **Axis 17 (Simulation Context):** simulation or scenario identifier (marks synthetic or hypothetical context) ⁷.

All former dimensional references in Ω have been **remapped** to these axes. In other words, any knowledge coordinate now explicitly populates the corresponding axis fields instead of using a custom schema. This ensures one-to-one alignment with the 17-axis schema defined in the UKG's formal documentation. The UKG's nodes and relationships are *indexed by these axes*: each knowledge node N_i carries values for axes 1–13 (describing *what/where/when* the knowledge is) and meta-axes 14–17 (describing *how valid/uncertain/ethical* it is and whether it's simulated) ¹ ². Pillars, sectors, and other categorical dimensions are thus encoded directly in the coordinate, and semantic similarities between knowledge items can be gauged by

comparing these axis values. For instance, two nodes sharing the same Pillar (Axis 1) or similar Honeycomb links (Axis 3) indicate a semantic affinity in that dimension. Likewise, persona roles (Axes 8–11) attached to a node identify the perspective or expertise from which that knowledge was generated or is most relevant

5 .

Formally, we define a **spatial mapping function** Φ that projects any input (query or knowledge item) into the 17-dimensional knowledge space:

$$\Phi : \text{Input} \rightarrow (x_1, x_2, \dots, x_{17}) \in \Omega.$$

This function encompasses NLP parsing and ontology mapping (Knowledge Algorithm KA-5 in the system) to assign the appropriate Pillar, Sector, etc., to the input 8 . For a given query Q , $S(0) = \Phi(Q)$ yields the initial structured representation K (the coordinate in axes) for that query 8 . *Example:* a query about a defense contract clause might map to $K = (1.01, 2.92.1.1, 3.FAR \leftrightarrow DFARS, 4.2.1.3, 5.1127, \dots, 12.US.DC, 13.2025.Q4, 14.0.95, 15.High, 16.Compliant, 17.SimID-7842)$ as shown in an example UKG run 9 . Here Axis 1 indicates an *Acquisition Regulations* pillar, Axis 3 links *FAR* to *DFARS* (showing related rule sets), Axis 5 picks a specific clause node, and the meta-axes show an initial confidence of 0.95, a High cyber-risk context, etc. All such coordinates are fully traceable and auditable within the system’s knowledge graph.

Dynamic Knowledge Mapping in UKG

Dynamic knowledge mapping refers to the system’s ability to place and update knowledge points in this multi-axis space on the fly as context evolves. The mapping function Φ is applied not just at query initialization but throughout the reasoning process whenever new intermediate knowledge is generated. The knowledge space is treated as a mathematical field $K(x, t)$ that varies over the axes $x = (x_1, \dots, x_{17})$ and over time t (representing the *temporal state* of knowledge) 10 . We can describe the **Universal Knowledge Graph state** as a time-dependent function:

$$**KG**(t) = \sum_i (N_i \cdot W_i) + \iint K(x, t) dx dt,$$

where N_i are discrete knowledge nodes with weights W_i (e.g. importance or confidence), and the integral term represents the continuous **knowledge distribution** over the axes and time 10 . In essence, the sum captures the *static* graph content and the double integral captures its *evolution* and diffusion over the 17 axes and time. For the 17-axis system, this can be generalized in a tensor form: $KG(t) = \sum_{j=1}^{17} T_j(t)$, where each $T_j(t)$ is the projection of the knowledge state onto axis j 11 . This formalism means each axis contributes a component to the overall knowledge state; for example, one can project the graph onto Axis 1 (Pillar) to see the distribution of knowledge across fundamental domains at time t , etc.

More concretely, when a new query arrives or new information is considered, the system dynamically **assigns coordinates and updates local knowledge** in RAM rather than relying on a static database. The function Φ is applied to identify which coordinates in Ω are relevant, and those “points” are used to fetch or generate knowledge. We incorporate a **knowledge distribution PDE** to model this process 12 :

$$K(z, t) = \int \phi(z, s) \psi(s, t) ds + \lambda \nabla^2 K. \quad (1)$$

Here z denotes an axis vector position and t time. The kernel $\phi(z, s)$ is essentially the spatial mapping function (like Φ above) used in a continuous sense – it maps a latent knowledge source at state s into the multi-axis space (e.g. projecting an input text into axis coordinates) ¹³. The term $\psi(s, t)$ represents the **temporal evolution operator** from state s to time t , which encapsulates how knowledge validity or relevance changes over time (including *entropy-based decay or growth*) ¹³. The second term $\lambda \nabla^2 K$ is a diffusion term (with λ a learning-rate parameter) that allows knowledge to “spread” or propagate locally across adjacent coordinates – for instance, if one node is updated, neighboring nodes along some axes can also be adjusted gradually, akin to heat diffusion on a manifold ¹⁴. This models **recursive learning** in the knowledge space: as the system discovers new implications or refinements, it updates the surrounding knowledge landscape in RAM immediately, without needing an explicit external retraining step ¹⁴. In practical terms, it means the system can perform **infinite in-memory recursion**, iteratively refining answers by updating knowledge coordinates and relations, achieving extremely low-latency self-updates (sub-millisecond for thousands of micro-adjustments) ¹⁴.

Notably, dynamic mapping is leveraged inside the persona simulations as well. Each persona’s internal reasoning can **generate new knowledge nodes on the fly** (hypotheses, intermediate conclusions) which are given coordinates and merged into the working graph. For example, when a persona recalls a specific regulation detail from the model’s training, the system “unlocks” that fact and assigns it a coordinate (with appropriate Pillar, Node ID, etc.), thereby treating it as a retrieved node in the graph ¹⁵ ¹⁶. This coordinate-tagging of even transient knowledge enables traceability and on-the-fly expansion of the knowledge graph during a query. Every newly mapped point can be validated or cross-referenced via its axes (e.g., ensuring a law clause pulled in by the Regulatory persona carries Axis 10 and Axis 6/7 tags linking it to other laws for consistency checks).

Because the mapping is dynamic, **no persistent storage lookup is needed for recall** – the language model’s *parametric memory* (pretrained knowledge) plus the context is used to hallucinate or reconstruct the needed facts, and axes coordinates are assigned to those facts to integrate them into the structured space ¹⁷ ¹⁸. This technique essentially transforms the model’s raw knowledge into a structured form on demand, creating what amounts to a temporary knowledge graph in memory. All of this is done within the FROST in-memory simulation stack, avoiding external API calls or disk I/O. *In summary*: mapping function Φ allows any piece of information to be positioned in the 17-axis space, and Equation (1) governs how these positions evolve and interconnect over reasoning time. The result is a **living knowledge space** where each reasoning step can expand or adjust Ω , and those changes feed into subsequent steps (a form of *online recursive learning* of context).

Structured Memory Recall Mechanism

With knowledge indexed in a 17-axis coordinate system, the system uses **structured memory recall** to fetch relevant information for a query. Instead of unstructured text search, the query’s coordinate $K = \Phi(Q)$ serves as a key into the UKG: it specifies which slices of the multidimensional space are likely relevant. The runtime **automatically retrieves the subgraph of UKG nodes whose coordinates match or are nearest to K** , subject to axis-specific similarity thresholds. In Phase 7’s execution engine, this is implemented as the “Knowledge Graph Loading” step, which takes the coordinate and pulls all nodes/edges in that vicinity into RAM ¹⁹. Formally, we define a similarity function $\text{sim}(K, N_i)$ between the query

coordinate and any node N_i 's coordinate. One simple formulation is to treat each of the first 13 axes (core knowledge axes) as a dimension for matching and require each to be within some tolerance for the node to be relevant:

$$\text{Relevant}(Q, N_i) = 1 \iff \forall j = 1..13 : d_j(K_j, (N_i)_j) < \varepsilon_j,$$

where $d_j(\cdot, \cdot)$ is a distance function on axis j (exact match or a metric for hierarchical indices), and ε_j is a threshold tolerance for that axis. In practice, this can be relaxed into a weighted similarity score across axes. For example, define local similarity s_j on each axis (e.g. $s_j=1$ if K_j and $(N_i)_j$ share the same category or fall in the same range, otherwise a value in $(0,1)$ decreasing with distance). Then an overall similarity could be a weighted sum:

$$\text{sim}(K, N_i) = \sum_{j=1}^{13} w_j s_j(K_j, (N_i)_j),$$

with weights w_j emphasizing more important axes for the query (these can be determined by which fields the query explicitly specifies or context). The system sets a threshold θ such that all nodes with $\text{sim}(K, N_i) \geq \theta$ are retrieved as relevant memory. This process yields a **focused subgraph** of nodes and facts, dramatically reducing search space by using the multi-axis index. (For instance, if Q is about a DoD contract regulation in 2025, the system will retrieve nodes with matching Pillar "Acquisition Regulations", Sector "Defense", relevant Branch for the contract clause, and time near 2025Q4, etc., while ignoring nodes outside those slices.) This occurs in runtime Phase 6–7 automatically, as indicated by Layer 2: Knowledge Graph Loading, where the math is $KG(t=0) = \sum N_i \cdot W_i + \int K(x, 0) dx$ — meaning the initial knowledge state is composed of weighted relevant nodes plus any continuous background knowledge distribution at the start ¹⁹.

Because each knowledge item carries meta-axes 14–17, the recall mechanism can also filter or flag based on those. For example, the system might **exclude nodes whose Confidence (Axis 14) is below a certain floor**, or adjust the weight W_i of each node by its confidence during retrieval ²⁰. It can also take into account Axis 16 (Ethics/Compliance) tags: e.g. if a node is marked *non-compliant* or ethically biased, a policy might prevent it from being used unless explicitly needed. In essence, the axes-based indexing not only finds data by topic and context, but also by *quality* and *suitability*. The result is a highly structured recall: the model "remembers" facts by reconstructing them from its training corpus and immediately indexing them with coordinates, as described above ¹⁷. Each persona's 7-part simulation (job role, training, rules, etc.) functions as a **guided memory probe** into the model's latent knowledge, causing it to surface specific pieces of information that align with that persona's expertise ²¹. Those pieces are slotted into the axes framework (often as transient nodes labeled by source, domain, time, etc.), making the recall *explicit*.

To illustrate, when the Regulatory Persona "recalls" a clause from FAR 52.204-21, the system maps that memory to an Axis 5 Node (the clause itself), attaches Axis 6/7 links (perhaps mapping it to related DFARS rules), and tags it with Axis 10=Regulatory Persona and Axis 14=high confidence (since it's a direct quote of law) ¹⁶ ²². This recalled node is then available to all personas for reasoning, just as if it had been stored in a database and retrieved. The **structured memory recall** is therefore the process of *populating the active knowledge subgraph in real-time* using the axes as keys. The conversation logs highlight this as a key innovation: the model leverages its training data (certifications, textbooks, regulations it was trained on) by "embodying" a persona, and through recursive self-application it **maps latent knowledge into a**

structured form that the reasoning engine can use ¹⁷ ²¹ . This approach is markedly different from traditional retrieval-augmented generation (RAG); rather than hitting an external vector store, the system creates a vector (coordinate) for the query and *internally* recalls what it needs, effectively compressing massive texts into a coordinate reference ²³ ¹⁸ .

All recalled knowledge points, being coordinate-tagged, come with provenance and can be traced. The traceability is “built-in”: every step in reasoning is tied to an axis location, which is why we can audit the source of any inferred fact by looking at its coordinate (for instance, a persona’s statement can be traced back to its **education** sub-coordinate or a Pillar coordinate that indicates the source domain of that knowledge) ¹⁶ . This structured recall underpins compliance and verification in the system, since each piece of used memory can later be checked for correctness or bias by examining its axis labels (Axis 14 confidence, Axis 16 ethical compliance, etc.) and cross-referencing with known sources.

Recursive Learning and Temporal Dynamics (Arrows of Time)

The Quad Persona System operates in a **recursive learning loop** to refine answers, and this loop is closely tied to the temporal axis and the concept of **Arrows of Time**. After the initial reasoning pass with the four personas, the system can re-inject the results (or any unresolved issues) as new inputs, updating the knowledge state $K(x, t)$ and repeating the reasoning process. This iterative refinement continues until a stopping criterion is met (e.g. confidence above a threshold and low contradiction entropy). In formal terms, if we label each full pass through the personas and arbitration as an iteration n , the refinement can be seen as seeking a fixed point where the validation metric $V(n)$ (defined below) converges to a high value. Each iteration n updates the knowledge distribution $K^{(n)}(x, t)$ (via Equation 1’s mechanism) and thereby influences the next iteration $K^{(n+1)}(x, t)$. The *deep recursive learning* strategy (sometimes referred to in shorthand as “deep **curative** learning” in the project logs) essentially means the model uses its own output as feedback to improve itself, all within memory ²⁴ . The FROST architecture supports this by snapshotting the state and looping internally without external calls, which is why the system can simulate, for example, a 12-step refinement workflow in a few seconds to boost confidence from, say, 95% to 99.5% ²⁵ .

A critical aspect of recursion is **temporal parameterization**. The reasoning doesn’t happen in a vacuum; it is aware of *time* (Axis 13) in multiple ways. First, knowledge is time-stamped. The query’s coordinate includes a time component (e.g. “2025.Q4”), and retrieved nodes also have time ranges or points. The system aligns them such that the answer will be valid for the query’s temporal context. If the query involves projecting into the future, the system must simulate those future knowledge nodes because none exist in the static graph beyond the present. This is where the **Arrows of Time (AoT)** notion comes in: it acknowledges that we have reliable records of the past but not of the future ²⁶ . The **thermodynamic arrow** and **psychological arrow** of time mean that entropy increases and we “remember” only past states, not future ones ²⁷ ²⁶ . In practical terms, the Quad Persona System will not pull information from beyond the current time t_0 (it has no direct data from the future, which would violate causality and data availability). Instead, if future insight is needed, it uses a *Temporal Simulation* approach: leveraging Axis 13 along with special knowledge algorithms and even “future personas” to **hypothesize future knowledge** in a controlled way ²⁸ ²⁹ .

For example, suppose the Knowledge Persona identifies that a technological breakthrough (not yet achieved as of 2025) would change the answer. At Layer 5 of the simulation, a **Temporal Breakthrough Mapping** routine (KA-38/43) might be invoked: it will generate a hypothetical node at a future time (say, Axis 13 = 2030) representing the breakthrough, and mark it with Axis 17 (Simulation) as a hypothetical

insertion ²⁸ ²⁹ . The system then *tests* the scenario with that future node present, effectively asking “if this future knowledge were true, how would it reduce the entropy of the solution?” ²⁹ . This approach is guided by the arrow of time: it moves **forward** to simulate possibilities, but only in the context of increasing entropy (e.g., requiring that any future technology be consistent with the trend of progress, not magically violate physical laws). The **entropy** concept is quantitatively in the reasoning as well – for a hypothetical future injection to be considered viable, it must reduce the overall “entropy” (uncertainty/conflict) of the answer and yield a high confidence when the personas re-evaluate the scenario ³⁰ . If it does, the system notes the conditions needed for that scenario (e.g., “a breakthrough by 2031Q2 in muon-catalyzed fusion is required to achieve the goal” ³¹). All these steps abide by temporal logic: knowledge is only *added* moving forward (one cannot retrieve unknown future facts, only simulate them under assumptions), and any such additions are clearly labeled on Axis 17 and Axis 13.

Mathematically, the **temporal evolution operator** $\psi(s, t)$ in Equation (1) encodes this one-way progression. It may include a decay factor for outdated information (knowledge loses weight $\$W_i$ as it gets older if not refreshed, reflecting entropy increase or relevance decay), or a growth term for accumulating evidence over time ¹³ . The $\int T(t) dt$ term in the validation metric (discussed next) also integrates the effect of time over the reasoning process ²⁰ . The **Arrows of Time influence** thus appears in: (a) the **data availability** (no future data except via simulation), (b) the **entropy budgeting** (making sure each step forward in time does not decrease entropy arbitrarily, in line with physical reality), and (c) the **time-aware confidence adjustments** (older info might be less trusted, future projections carry higher uncertainty captured as higher entropy/risk Axis 15). In summary, the system is aligned with the arrow of time by design: *it remembers the past, is aware of the present, and carefully simulates the future with increasing uncertainty*. Each persona’s analysis also respects this; e.g., the Knowledge Persona’s “Epistemic Forecaster” sub-role might explicitly call out where current theory runs into unknowns beyond the current time horizon ³² . This temporal awareness is critical for ensuring that answers are time-bounded or include validity ranges (the refinement Step 8 adds *temporal validity tagging* to the output ³³). For instance, an answer might be tagged “Valid 2026–2030, re-check afterwards” if based on assumptions that could change ³³ . All of these measures ensure that the **temporal parameter $\$t$ is integrated consistently with the Arrows of Time** concept – the system’s knowledge state $\partial K / \partial t$ follows an irreversible trajectory, and our reasoning pipeline leverages that by only extrapolating forward with explicit caution.

Integration Function Ψ for Persona Outputs

After each persona has processed the query and contributed its perspective, the Quad Persona System must integrate these into a single coherent answer. We define an **integration function** Ψ which takes the set of persona outputs and produces a unified result along with a confidence score. In the 17-axis architecture, Ψ is designed to be *axis-aware*: it uses the meta-axis values (confidence, risk, etc.) and can apply policy-based weights to different personas. A rigorous way to express this integration is via the **validation metric** $V(n)$, which combines various factors into one score ²⁰ . Drawing from the formal documentation, one formulation is:

$$**V(n)** = \alpha \prod_i (p_i \cdot c_i) + \beta \sum_i (r_i \cdot q_i) + \gamma \int T(t) dt \cdot CA. \quad (2)$$

This equation ²⁰ merits explanation: it integrates **provenance and confidence** (the first term), **risk and impact** (second term), and **temporal & compliance factors** (third term) for the collective output. Each persona $\$i$ (for $\$i =$ Knowledge, Sector, Regulatory, Compliance) provides: a confidence $\$c_i$ (Axis 14

value, how strongly this persona supports its answer), and may have an associated provenance or priority factor p_i (which could represent the reliability of that persona's sources or an explicit policy weight for that persona). The product $\prod_i (p_i c_i)$ effectively says *all personas must be confident with good provenance for $V(n)$ to be high* – if any c_i is low or a persona lacks support (low p_i), the product drops, reflecting a consensus requirement ²⁰. The second term involves each persona's risk assessment r_i (Axis 15, how much uncertainty or risk that persona sees) and perhaps q_i , which can be interpreted as an *impact factor* or significance weight of that risk (for example, if Regulatory Persona flags a legal risk, q_i might quantify the severity or scope of that risk). The sum $\sum_i (r_i q_i)$ aggregates the risks from all personas ²⁰. β is a tuning coefficient that governs how strongly risks weigh against the confidence product. The final term $\int T(t) dt \cdot CA$ represents any **temporal penalty or adjustment** (integrating a time-based trust function $T(t)$ over the relevant interval) multiplied by a **Compliance Audit factor** CA ²⁰. The CA might be a binary or graded factor coming from an ethics/compliance module (Axis 16): for instance, $CA = 0$ if the combined answer violates a hard compliance rule, or $CA = 1$ if fully compliant, or a fractional score if partially. The γ coefficient sets the relative importance of time/compliance considerations in the overall metric.

Function $V(n)$ is essentially the **score of the answer at iteration n** . The integration function Ψ can be seen as producing the final answer *and* implicitly computing this $V(n)$ as the confidence of that answer. In operational terms, Ψ uses a process of **persona arbitration and weighting**: it takes each persona's proposed answer content and their meta-scores $(c_i, r_i, p_i, \text{etc.})$, and performs a weighted aggregation guided by Equation (2). The **axis-weighted confidence scoring** is evident in the $\alpha \prod_i (p_i c_i)$ term – here axis 14 (confidence) and provenance factors directly influence the final confidence ²⁰. For example, if all four personas independently reach similar conclusions with high confidence (say $c_8=0.98, c_9=0.96, c_{10}=0.99, c_{11}=0.98$) and strong sources (p_i near 1), the product term will be high ($\sim 0.9+$). If even one persona is uncertain (c_i low) or has weak grounding (p_i low), the product drops, lowering $V(n)$ significantly – thus the system naturally demands consensus or at least no serious dissent for a high confidence ²⁰. The second term ensures that if any persona raises a serious risk (high r_i weighted by large q_i), that will subtract from the overall score unless mitigated. In other words, a high-risk finding by (say) the Compliance persona will push $V(n)$ down, reflecting lower final confidence until that risk is addressed or acknowledged. The third term ensures that if the answer's validity is short-lived or far-future dependent (large temporal uncertainty) or if it fails compliance checks ($CA < 1$), the score is penalized accordingly ²⁰.

This formalism directly corresponds to using the **meta-axes as integration drivers**. Axis 14 and 15 values from each persona feed the first two terms, Axis 13 and 16 feed the last term (with $T(t)$ capturing any temporal decay or urgency and CA capturing ethical/compliance alignment) ²⁰ ³⁴. The integration function Ψ outputs a *final answer* (synthesizing the textual reasoning from personas) and attaches a final confidence score (which is essentially $V(n)$ normalized, e.g. 98.2% in one example) and an entropy or uncertainty measure (which could be derived from $1 - V(n)$ or specifically the entropy observed in persona conflicts) ³⁵. For instance, in a cold-fusion query example, after arbitration the system reported *Confidence 0.982, Entropy 0.037* ³⁵. These numbers reflect the outcome of such a Ψ calculation – high confidence and low entropy because all four personas largely agreed, and any remaining minor conflicts were resolved by policy (Regulatory persona's safety concerns were prioritized) ³⁶.

Importantly, **policy-governed persona priority** is built into Ψ . Policies can influence the integration in a few ways: (1) through the weights p_i or q_i – for example, if a policy says "Regulatory compliance is paramount," we can effectively increase the weight of the Regulatory persona's inputs by giving it a higher

p_{Reg} or amplifying q_{Reg} for risks it raises. Then Equation (2) will more heavily penalize non-compliance, ensuring the final answer favors the Regulatory persona's stance in case of conflict. (2) through the CA factor or direct conditional rules – e.g., a policy might stipulate $CA=0$ (reject answer) if Compliance persona identifies any *critical* violation. This would zero-out the third term and heavily reduce $V(n)$, possibly forcing another iteration to fix the issue. In practice, the arbitration step in Layer 7 applies a set of rules to resolve disagreements: e.g., “*safety-related regulatory concerns override optimistic technical projections*”³⁶. This was seen when *Regulatory Persona wins arbitration over a Safety Clause priority* in a conflict, meaning the policy favored the regulatory view for final output³⁶. We can incorporate such rules by having Ψ_i check conditions like: *if persona j flags a high-risk (r_j above threshold) in a high-priority category (per policy), then choose persona j's conclusion as dominant or require mitigation*. In a mathematical sense, this can be represented as constraints on the optimization of $V(n)$ or conditional modifications to the weights. For example, introduce a very large β for certain risk types to strongly penalize them, or set α to zero if any persona veto condition triggers (making $V(n)$ drop regardless of others).

To give a simplified pseudocode of the **integration and refinement procedure** consistent with Phase 7 runtime:

```
# After persona outputs are collected...
inputs = { (A_i, c_i, r_i, p_i) for i in [Knowledge, Sector, Regulatory,
Compliance] }

# Policy weighting for personas (could be dynamic based on query or config)
for each persona i:
    if policy.prioritize(persona_i):
        p_i *= policy.weight_boost # emphasize this persona's provenance/
priority
    if policy.veto_condition(persona_i):
        return persona_i.adjusted_answer, final_confidence=low # early return
or mark for refinement

# Compute integrated confidence via Equation (2)
conf_product = 1
for i in inputs:
    conf_product *= (p_i * c_i)
risk_sum = 0
for i in inputs:
    risk_sum += (r_i * q_i) # assume q_i derived from context or persona
impact
time_penalty = integrate(T(t) from t0 to t_query) # e.g. decay factor for
outdated info
compliance_factor = evaluateCompliance(inputs) # 0 to 1

V = alpha * conf_product + beta * risk_sum + gamma * time_penalty *
compliance_factor

# Derive final answer text (e.g. combine A_i with justification)
```

```

final_answer = synthesize(A_Knowledge, A_Sector, A_Regulatory, A_Compliance)
final_confidence = normalize(V) # scale V(n) to 0-1 range if needed

if final_confidence < required_threshold:
    # If confidence or compliance is insufficient, refine (recursive loop)
    refine_query = analyze_gaps(final_answer, inputs) # identify which persona
    disagreed or what risk present
    update_context(refine_query) # e.g. add a hypothetical node or adjust
    some assumption
    goto personas_simulation_step # run another cycle with new context
else:
    return final_answer, final_confidence

```

This pseudocode sketches how persona outputs are merged, with policy checks and Equation (2) guiding the scoring. The **synthesis** of the final answer (the natural language output) typically involves merging the rationales of each persona and resolving any contradictions. For instance, if Knowledge persona says “technically viable” but Regulatory says “not allowed without X”, the final answer might state the viability along with the regulatory caveat. The arbitration mechanism (entropy minimization) will choose phrasing or conclusions that respect high-priority constraints – e.g., including the compliance requirement so that the Regulatory persona’s concern is satisfied ³⁶.

Once Ψ produces an answer and if $V(n)$ meets the confidence threshold (often set around 99.5% for mission-critical answers ³⁷), the system stops. Otherwise, it enters another iteration: perhaps an optional persona is activated (if high entropy persists), or the personas are prompted to reconsider with the new information (like “Regulatory persona, assume the new mitigation Y is in place, does that resolve your concern?”). This **recursive learning loop** continues until convergence, at which point the answer is finalized with a high confidence stamp and rich trace metadata. All of this is aligned with Phases 6–8 runtime design: Phase 7’s scheduler orchestrates these persona activations and checks via YAML-defined processes, ensuring each layer (mapping, retrieval, persona reasoning, arbitration, etc.) occurs in order ³⁸. The **10-layer simulation stack** (as described in the documentation) corresponds to our formulation: Layer 1–2 perform Φ and knowledge recall, Layers 3–6 run personas and initial arbitration, Layer 7 (and possibly 8) run the Ψ integration and any hypothetical extensions, and Layers 9–10 finalize output and logging ³⁸. The Quad Persona System’s activation in Phase 7 is timed such that core personas always engage in layers 3–6, and optional personas (e.g. counterfactual or future personas) are only triggered in Layer 5+ if the entropy after core deliberation is above a threshold ³⁹ ⁴⁰. This scheduling is inherently supported by our formulation: the entropy (conflict) check comes from the same integration process (low $V(n)$ or a large risk term would indicate high entropy, prompting additional personas or iterations).

In conclusion, the updated mathematical formulation tightly integrates the Quad Persona Engine with the 17-axis knowledge schema. We map every component of the personas’ reasoning to coordinates in the UKG (via Φ), use those coordinates to structure memory recall and ensure traceability, and combine persona outputs with an axis-informed integration function Ψ that yields a confident, policy-aligned answer. Temporal dynamics (Arrows of Time) and recursive self-refinement are explicitly accounted for through the time axis and iterative update equations, making the system **self-consistent over time and knowledge expansions**. All notation and function names here (Ω , Φ , Ψ , $V(n)$, etc.) follow the evolving formal documentation of the 17-Axis System, and this formulation is meant to be pasted directly into the technical report (Phases 6–8) as the analytical description of how the Quad Persona System operates within the full

UKG framework. The result is an architecture where four expert personas, each with structured sub-roles, collectively reason over a shared, axis-indexed knowledge space, producing answers that are not only correct, but also **explainable, time-aware, and compliant by design** ¹⁶ ²⁰ .

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ²⁸ ²⁹ ³⁰ ³¹ ³²
³³ ³⁴ ³⁵ ³⁶ ³⁷ ³⁸ ³⁹ ⁴⁰ 17axis.txt
file://file-7HjxCe5SqRbVKpsfqmyyD6
²⁶ ²⁷ 2025-10-4 1-2-7-Arrows_of_time_explanation.txt
file://file-KpEqmwaY2UTr5e7uPp292e